Software Architecture Specification (SAS)

Projekt 6: API für den Digitalen Produktpass (DPP) im BaSyx Framework

Customer

Name	Mail
Markus Rentschler	rentschler@lehre.dhbw-stuttgart.de
Pawel Wojcik	pawel.wojcik@lehre.dhbw-stuttgart.de

Task Description

> The DIN EN 18222 “Digital Product Passport – Application Programming Interfaces (APIs) for lifecycle management and product passport searchability” defines a REST API that must be implemented as part of this assignment within the BaSyx framework, including both backend and frontend components. The detailed task description can be found here.

Change History

Version	Date	Author	Comment
1.0	2025-11-19	Noah Becker	First architecture specifications
1.1	2026-05-08	Noah Becker	Substantive changes
1.2	2026-05-14	Noah Becker	Implementation of proposed changes

---

Table of Contents

---

1. Introduction

1.1. Purpose and Scope

A **Digital Product Passport (DPP)** is a standardized, machine-readable record that captures a product's lifecycle data — its materials, carbon footprint, technical specifications, and condition. International standards such as **DIN EN 18222** define the APIs required to create, query, and manage these passports across systems and organizations. Content (Submodels) of the DPP is standardized by the **IDTA-02035-X**-Standard, which defines the 7 relevant Submodels for a DPP (**-1:** NamePlate; **-2:** HandoverDocumentation; **-3:** CarbonFootprint; **-4:** TechnicalData; **-5:** ProductCondition; **-6:** MaterialComposition; **-7:** Circularity).

This project implements those standards within **BaSyx** — an open-source platform built around the Asset Administration Shell (AAS) model, which provides the underlying data representation for DPPs. The result is a REST API and an accompanying frontend view that make DPP data accessible to both end users and integrating systems, hosted as independent microservices alongside the existing BaSyx infrastructure.

This SAS defines the architectural design of the Digital Product Passport (DPP – dt. *Digitaler Produkt Pass*) software, including API endpoint specifications, frontend integration within the BaSyx WebUI, component responsibilities, and deployment considerations.

The SAS defines how the system fulfills the functional and non-functional requirements defined in the *Software Requirements Specifications (SRS)*.

Scope:

The architecture described here covers frontend, backend and API specifications of the software segment within the DPP API and Frontend views.
The following areas are considered out of scope: general BaSyx software architecture.

1.2. System Overview

The system comprises two primary components: the *DPP Viewer* and the *DPP API*.

- ***DPP Viewer*** — A web viewpoint that presents DPP-related AAS submodels in a clear, responsive UI. It emphasises usability and maintainability and integrates into the BaSyx WebUI.
- ***DPP API*** — RESTful API endpoints exposing DPP data and submodel elements to developers and integrators, working by its own (docker) container service. It offers JSON responses, query/filter endpoints, and machine-readable API documentation (OpenAPI/Swagger).
- ***DPP Registry*** — An administrative web interface for managing DPP instances associated with Asset Administration Shells. It allows users to create and delete DPPs, review their metadata and submodel references, and navigate to the AAS Editor for modifying the underlying submodel content. It integrates into the BaSyx WebUI and targets administrative use cases rather than end-user product viewing.

Key capabilities:

- Visualisation of DPP-related AAS submodels for end users
- Editable view of DPP instances of an AAS
- Programmatic access to DPP data via REST endpoints
- Easy-to-use API documentation through Swagger/OpenAPI
- Integration into the BaSyx WebUI

The system follows a microservices (Docker) architecture.
Primary technologies include a Vue.js-sided Frontend, Java SpringBoot Backend, and a pipeline server deployment.
External dependencies include the BaSyx Backend Services *BaSyx AAS Environment*, *BaSyx AAS Registry*, *BaSyx Submodel Registry* and *BaSyx AAS Discovery*.

2. Architectural Overview

The BaSyx DPP API operates as part of a broader environment that includes external users, services, and data sources.

2.1. System Context

This section provides a Black-Box perspective, showing the system's boundaries, inputs, outputs, and primary interactions.

Purpose: To define what the system communicates with – *who* or *what* it depends on, and *what* depends on it.

Context Description: The system receives requests, processes this data through the exisiting AAS API, maps these responses to the DPP requirements, and produces an API response.

External Entity	Type	Interaction / Data Flow	Communication Channel
End User	Human actor	Submits requests via UI and receives Feedback	Web UI / Browser
AAS Environment API	API	Provides necessary AAS content	REST API / HTTPS
Administrator	Human actor	Monitors, configures, and maintains the system	SSH

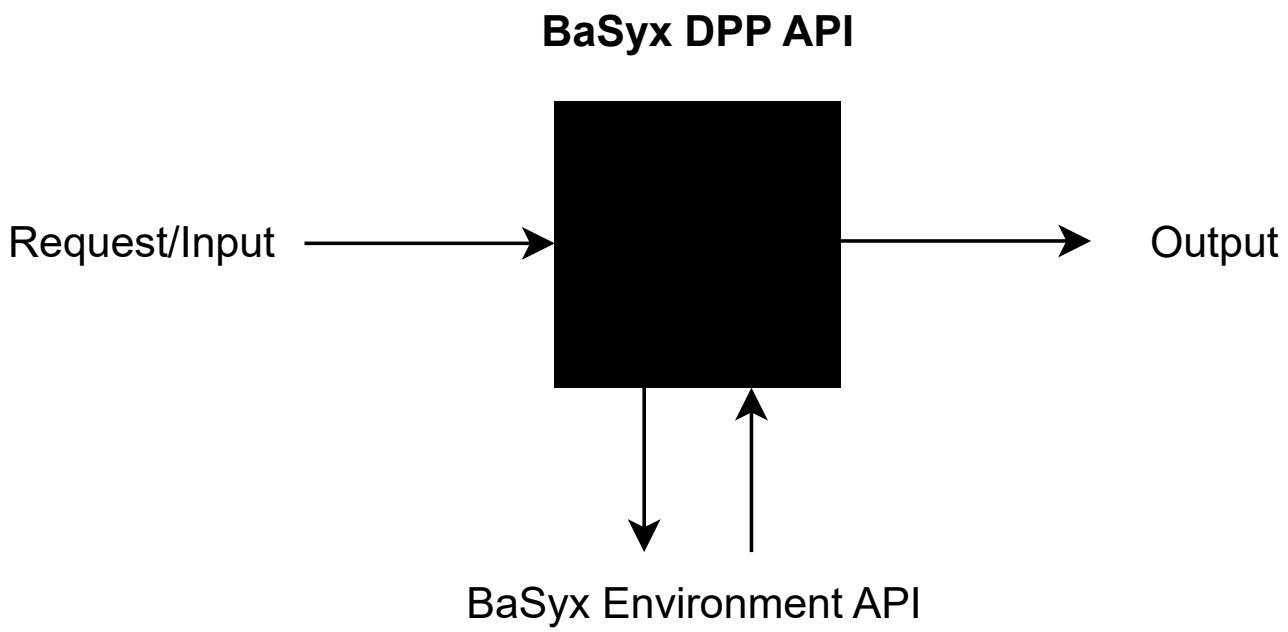

Figure 2-1 — System Context Diagram (Black-Box-View) of the BaSyx DPP API showing external actors and data flows.

2.2. Design Approach

This section outlines the architectural style, principles, and technologies used to implement the system.

Architectural Style
The system integrates into the existing BaSyx infrastructure, working microservice-based:

- **DPP Viewer *(Frontend)*:** The frontend integrates into the BaSyx WebUI as a web view, based as a module.
- **DPP API *(Backend)*:** The backend works by its own independent docker container, providing new API endpoints.

The given microservice architectural style emphasizes:

- **Service Isolation:** Each service runs it its own container.
- **Scalability:** Individual services can be scaled horizontally as needed.
- **Independent Deployment:** Updates can be applied to each component separately via CI/CD pipelines.
- **Loose Coupling:** Services communicate through well-defined HTTP APIs.

Logical Layers

Layer	**Description**	**Implementation** (more *here*)
Presentation Layer	Provides the user interface and handles client-side logic	Vue.js (JavaScript)
Application Layer	Implements the core business logic and RESTful API	Spring Boot (Java)
Data Layer	Manages persistent data and ensures data integrity	mongoDB
Infrastructure Layer	Handles routing, deployment, and orchestration	Traefik Reverse Proxy, Docker, GitHub Actions CI/CD

Architectural Principles

- **Separation of Concerns:** UI, logic, and data are clearly divided across independent services.
- **Loose Coupling / High Cohesion:** Components interact only via HTTP interfaces, maintaining clear boundaries.
- **Containerization:** All services are encapsulated as Docker containers for consistent runtime environments.
- **Automated Deployment:** GitHub Actions automates build, test, and deployment processes to ensure reliability and traceability.
- **Security:** Traefik enforces HTTPS routing, and Spring Boot handles authentication and role-based access control.
- **Scalability & Maintainability:** Each microservice can be updated or scaled independently without system downtime.

Technology Stack

Layer / Aspect	Technology	Purpose
Frontend	Vue.js (JavaScript)	User interface and interaction
Backend	Spring Boot (Java)	Application logic and API gateway
Proxy / Router	Traefik	Reverse proxy, SSL termination, routing
Containerization	Docker	Service packaging and isolation
CI/CD Pipeline	GitHub Actions	Automated build, test, and deployment

Design Justification

The chosen microservice-based architecture allows modular development and simplifies maintenance by separating concerns across independent components.
Using Docker ensures consistent environments across development and production.
Traefik dynamically routes requests between containers and provides secure HTTPS access.
The GitHub Actions pipeline ensures continuous integration and deployment, improving code quality and deployment speed.

3. Structural Views

This section provides the structural architecture of the BaSyx DPP API.

It presents the system at different abstraction levels to illustrate how the main subsystems are organized and how their internal components collaborate.
The Grey-Box view describes subsystem boundaries and interactions, while the White-Box view details internal components and class-level structure.

3.1. Grey-Box View

The BaSyx DPP API system is decomposed into multiple independent submodules to support scalability, maintainability, and modular development. Each submodule encapsulates a distinct responsibility and communicates through well-defined interfaces.
This decomposition enables parallel development, reduces coupling, and allows individual servies to be deployed and scaled independently.

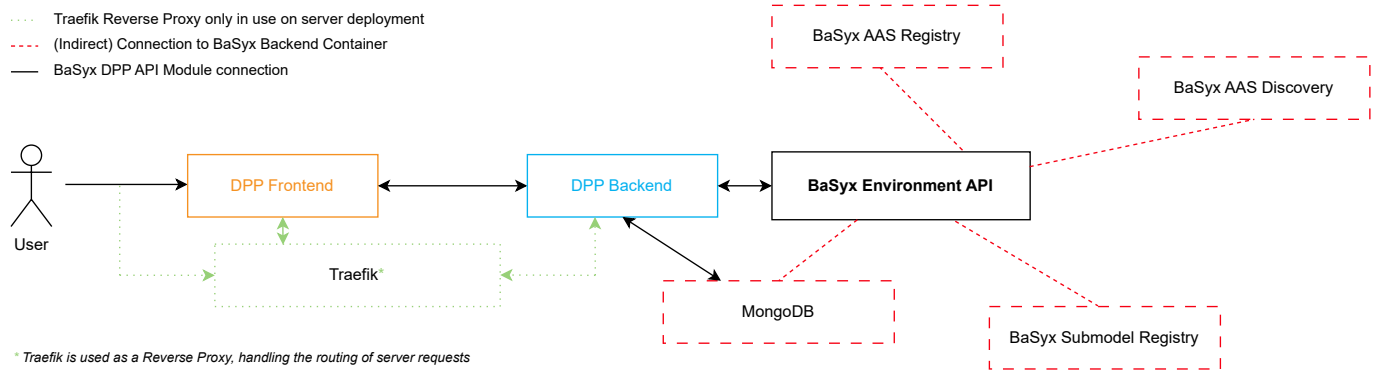

Figure 3-1 — Subsystem architecture overview of the BaSyx DPP (API), showing the central microservices and their integration points with existing BaSyx backend services.

Subsystem Responsibilities:

Submodule	Responsibility
DPP Frontend	User interface rendering, interaction handling, form validation, HTTP request flow
DPP Backend API	Business logic, request validation, domain mapping, REST endpoints
BaSyx Environment API	Provides access to persisted AAS-related product information and submodel data
Traefik	Routes incoming requests, performs SSL termination, and handles service discovery

> [!NOTE] The DPP Frontend is detailed further in the Frontend MOD.

> [!NOTE] Routing configuration, SSL termination, and Traefik setup are covered in the Routing MOD.

Subsystem Collaboration

The DPP Frontend communicates with the Backend through RESTful HTTP requests routed via the Docker internal networking. The DPP Backend retrieves product- and submodel-related information by internal queryies to the BaSyx Environment API, which connects to a MongoDB database. Traefik dynamically routes traffic based on

container labels, ensuring request isolation and secure HTTPS access.

Responsibility Boundaries

- Presentation concerns are strictly contained within the DPP Frontend.
- Domain logic, schema mapping, and validation remain inside the DPP Backend.
- Persistence and AAS service interaction are delegated to the corresponding API endpoints provided by the BaSyx Environment API.
- Infrastructure-level routing and TLS termination are handled centrally by Traefik.

Known Limitations

- The BaSyx Environment API is treated as a black-box dependency. Changes to its data schema may require adaptation layers in the backend.
- Runtime coupling exists with the BaSyx Environment availability; fallback strategies are limited in the current state.

3.2. White-Box View

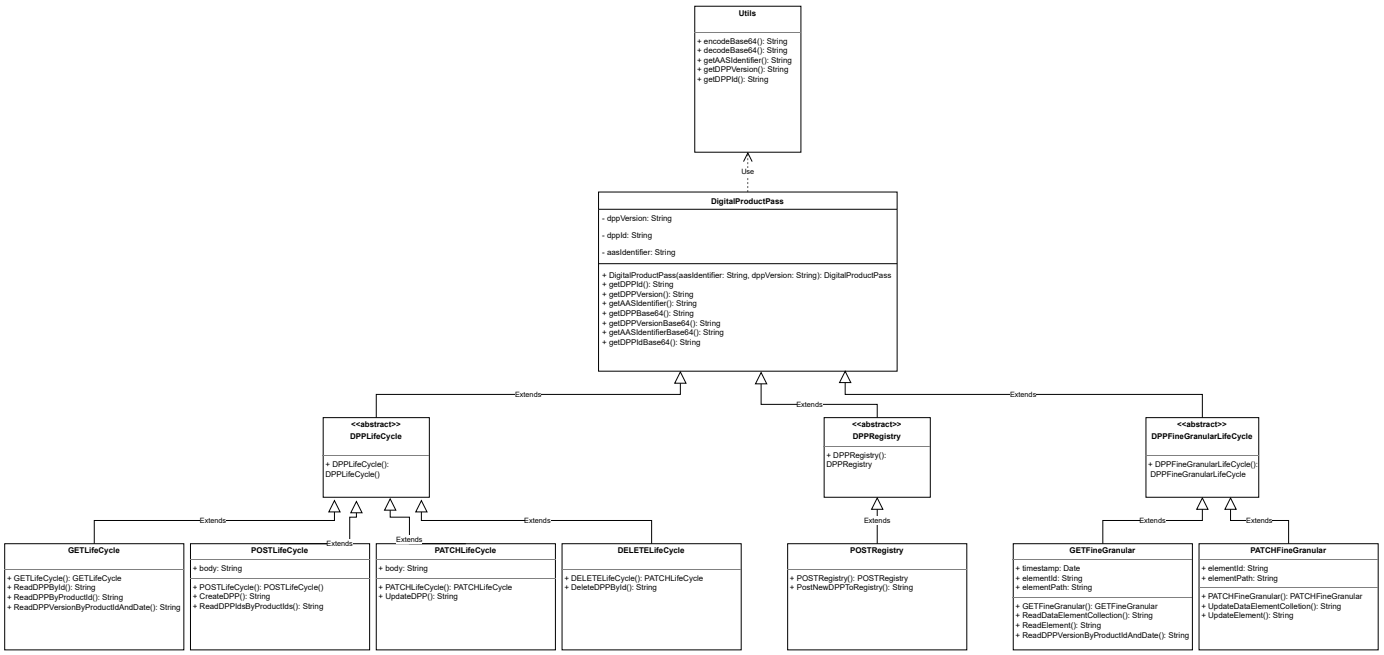

Figure 3-2 — White-Box View / UML diagram / Architectural structure for the DPP API Backend – Providing information about necessary classes and dependencies which will be included throughout the development. (Authors: Luca Schmoll & Fabian Steiss)

> [!NOTE] The backend implementation — including all controllers, data models, utility classes, and API endpoints — is documented in detail in the Backend MOD.

4. Behavioral Views

4.1. Communication Diagram

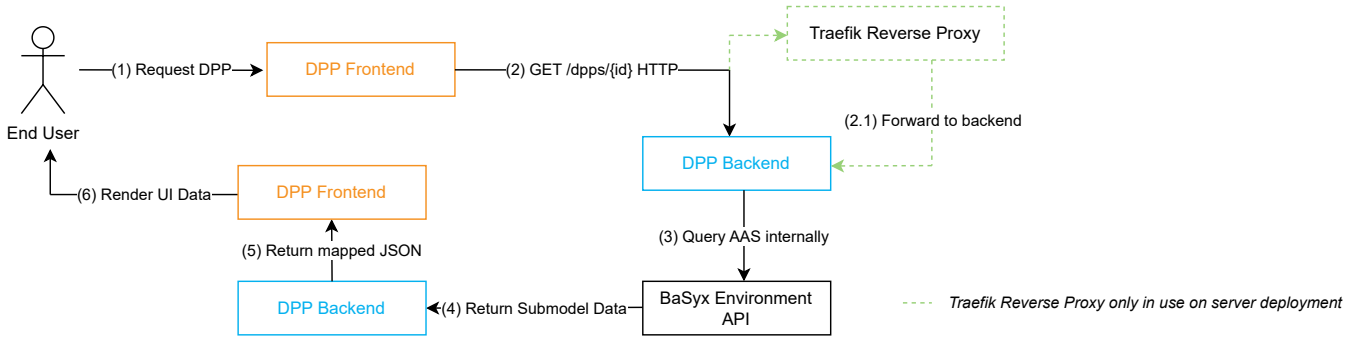

Figure 4-1 — Communication Diagram for the DPP Data Retrieval – Communication flow between user, frontend, Traefik, backend, and the BaSyx Environment API during a Digital Product Passport (DPP) request.

The user triggers the request via the DPP frontend. The DPP frontend calls the DPP backend service using RESTful HTTP requests. *Traefik routes the request to the correct container based on service labels. (On server deployment)*
The DPP backend internally queries the BaSyx Environent API endpoints to retrieve AAS and submodel data and then maps the result to the internal DPP schema. The processed data is returned to the DPP frontend as a JSON payload and finally rendered in the UI.

If the BaSyx Environment API endpoints are unavailable, the DPP backend returns a descriptive error response.

4.2. Sequence Diagram

The following sequence diagram provides a detailed view of the runtime behavior of the system when a user requests Digital Product Passport (DPP) data. It shows the chronological order of messages exchanged between the involved components and highlights the responsibilities of each subsystem during the request-response lifecycle.

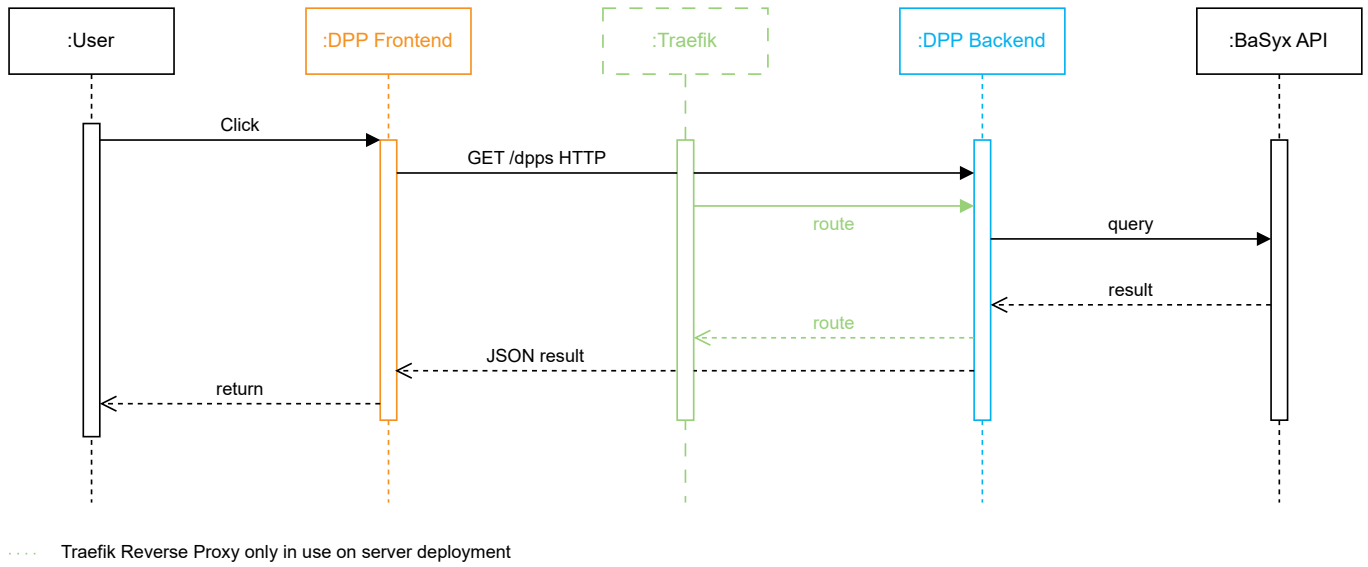

Figure 4-2 — Sequence Diagram for DPP Data Retrieval – chronological interaction between the user, DPP frontend, Traefik, DPP backend, and the BaSyx Environment API endpoints during a Digital Product Passport (DPP) request.

When a user initiates a DPP data request through the DPP frontend, the application sends an HTTP request to the DPP backend service via the Traefik reverse proxy. Traefik forwards the request to the appropriate DPP backend container based on routing rules. The DPP backend validates the request, queries the BaSyx Environment API endpoints for the required AAS and submodel information, and maps the received data to the internal DPP schema. Once processing is complete, the DPP backend returns the consolidated JSON response to the frontend, which then renders the corresponding product information to the user.

Rationale

This sequence demonstrates a clear separation of concerns: user interaction and rendering are handled by the DPP frontend, data orchestration and processing by the DPP backend, and AAS data retrieval by the BaSyx Environment API endpoints. This separation improves maintainability, testability, and supports the modular microservice architecture.

Known Limitations

If the BaSyx Environment API fails to respond, the backend returns an appropriate error message to the frontend to prevent partial or incorrect data from being displayed.

5. Data View

This view provides an architectural understanding of how data is represented, transformed, and exchanged between system components and external services.

5.1. Purpose

The purpose of the Data View is to describe the structure, flow, ownership, and constraints of the data that is relevant to the Digital Product Passport (DPP) system. It ensures transparency regarding data semantics and supports architectural decisions that impact maintainability, interoperability, and data integrity.

This view is architecturally relevant because the system relies on external data sources (BaSyx AAS infrastructure) and performs internal data mapping to deliver DPP-specific information to end users and API consumers. As a result, data consistency, interpretation, and transformation are central architectural concerns.

5.2. Data Model and Data Flow

The DPP backend maintains its own dedicated collection (`dpp-repo`) within the shared MongoDB instance. This store holds DPP records — but it does **not** duplicate AAS data. Instead, each record contains only the identifiers and references necessary to locate the corresponding AAS resources, keeping the DPP store lean and the AAS infrastructure as the single source of truth for product data.

DPP Record Structure:

Field	Description
`dppId`	Unique DPP identifier, derived from `productId` + creation timestamp
`productId`	Base64-encoded AAS identifier (`aasIdentifier`) referencing the AAS shell
`globalAssetId`	Base64-encoded global asset ID referencing the physical or logical asset
`createdAt`	Unix millisecond timestamp of DPP creation
`version`	DPP schema version
`submodels`	List of submodel references — each entry holds the submodel URI, its semantic name, and version; **no submodel content is stored**

A single shell document may contain multiple DPP entries in an array, representing different versions or lifecycle states of the same product's passport.

Data Flow Overview:

1. The DPP frontend or an external consumer requests DPP data from the backend.
2. The DPP backend retrieves the DPP record from `dpp-repo` (identifiers and submodel references only).
3. For each submodel reference, the backend issues API calls to the BaSyx Environment API to fetch the actual submodel element data.
4. The backend assembles the full DPP response — combining the stored metadata with the retrieved AAS content — and returns it as a unified JSON payload.

This separation ensures that AAS data is never redundantly stored: the DPP record acts as a reference index, and all product content is sourced live from the BaSyx Environment API at request time.

6. Technical Concept

6.1. DPP API Specification

> [!NOTE] The complete OpenAPI/Swagger specification for the DPP API is maintained in the API MOD.

The DIN EN 18222 (2025-08 Draft) specifies the necessary API-endpoints to enhance the searchability of DPPs and to support interactions throughout the lifecycle of a product's DPP. — It divides the DPP endpoints into 3 methods:

- **Life Cycle API (Main Methods):** Includes the main GET, POST, PATCH and DELETE functionalities
- **Registry API for Register:** Covers access to external methods of the EC Registry, in order to register a new DPP at the registry of the EC
- **Fine Granular API Operations of the Life Cycle API:** Provides fine-granular access to individual ElementCollections and Elements

Requests:
The important REST-API Calls **GET, POST, PATCH, DELETE**, the necessary parameters as well as request bodies are specified and should be included in the OpenAPI specification like described.

Respones:
Result objects should follow the described patterns *(in Table 13, 14, 15, 16)*. It is not outlined how the DPP Object itself should be structured!

Important notice:
As the DIN 18222 is still a draft, some things are listed contradictory multiple times. Also some information, e.g. about the definition of parameters, are unclear, therefore some assumptions will be taken.

Further description to the data approach, how and where the data needed is retrieved, is described in the following Chapter 6.2 – DPP Data Composition.

6.2. DPP Data Composition

Relevant Submodels:
The DPP of a product includes information about the following seven Submodels:

- Digital Nameplate — *(IDTA-02035-1)*
- Handover Documentation — *(IDTA-02035-2)*
- Product Carbon Footprint — *(IDTA-02035-3)*
- Technical Data — *(IDTA-02035-4)*
- Product Condition — *(IDTA-02035-5)*
- Material Composition — *(IDTA-02035-6)*
- Circularity — *(IDTA-02035-7)*

All relevant Submodels are stored in the AAS and need to be retrieved in order to return the DPP of a product.

Data flow — Example: DPP request:

Step (no.)	Step (title)	Description	Involved endpoint(s)
1	Request	User or third-party-application requests the DPP of a product	DPP Frontend / DPP Backend API endpoint
2	Data retrieval	DPP API Backend has to gather the necessary information about the submodels of the product	DPP API Backend BaSyx Environment API
3	Data mapping	Reducing the response-data to the DPP-relevant keys and map the response to the required DPP scheme	DPP API Backend
4	Response	Respond with the correct DPP data object	DPP API Backend Third-Party-Application
5^*^	Display data	Display the data visually in the DPP Viewer	DPP Frontend / User

** Only when request origins from the DPP Viewer*

Important considerations:

The less mapping, the more performance can be achieved. Therefore unnecessary mapping will be avoided, especially the POST and PATCH endpoints will follow the Environment API request body schemes so these can just be piped through without much adjustment.

The guidelines and specifications from the DIN EN 18222 should be respected meticulously, to achieve a standardized implementation of the DPP into the BaSyx Infrastructure. A first outline of the OpenAPI Specification based on the DIN EN 18222 can be found here *(local file)* and here *(online SwaggerUI)*.

The namings of the parameters in the DIN EN 18222 differ from the specifications in the BaSyx WebUI, the following table is designed to give a good overview about aliases, the purpose and the format of the parameters necessary for a DPP:

Parameter (DIN EN 18222)	Alias (BaSyx)	Purpose	Format	Example^*^
dppId	aasIdentifier + UNIX-Timestamp	Identify the AAS-Shell and its corresponding DPP	Input needs to be base64-encoded	https://dpp40.harting.com/shells/ZSN11778792078 *aHR0cHM6Ly9kcHA0MC5oYXJ0aW5nLmNvbS9zaGVsbHMvWlNOMTE3Nzg*
productId	*aasIdentifier*	Identify the AAS shell of the product	Input needs to be base64-encoded	https://dpp40.harting.com/shells/ZSN1 *aHR0cHM6Ly9kcHA0MC5oYXJ0aW5nLmNvbS9zaGVsbHMvWlNOMQ*
globalAssetId	*Global Asset ID*	Identify the physical or logical asset linked to the AAS shell	Input needs to be base64-encoded	https://pk.harting.com/?.20P=ZSN1 *aHR0cHM6Ly9way5oYXJ0aW5nLmNvbS8/LjIwUD1aU04x*
elementId	submodelIdentifier	Identify a specific submodel	Input needs to be base64-encoded	https://dpp40.harting.com/shells/ZSN1/submodels/CarbonFootprint/0/9 *aHR0cHM6Ly9kcHA0MC5oYXJ0aW5nLmNvbS9zaGVsbHMvWlNOMS9zdWJ*
elementPath	idShortPath *+ addition*	Take a specified path to an element in a submodel (dot-separated) **Implementation:** {submodelIdentifier}.idShortPath	submodelIdentifier needs to be base64-encoded Rest of idShortPath is "normal"	https://dpp40.harting.com/shells/ZSN1/submodels/CarbonFootprint/0/9.F *aHR0cHM6Ly9kcHA0MC5oYXJ0aW5nLmNvbS9zaGVsbHMvWlNOMS9zdWJ*

** This HARTING AAS-Shell is used for example data.*